Control Flow and Processor Control

COMS10015

Dr Tom Deakin

University of Bristol

Week 15

Architecture: Control Flow

These slides are based on material from Dan Page.

Motivation

The programs we write normally don't just execute a fixed sequence of steps: they have *conditional* statments (`if () {} else {}`).
They also are built from many functions, so we have to execute subsections of the sequence:

```
void do_this() { ... };
void also_this() { ... };
void then_this() { ... };

int main(void) {
  do_this();
  also_this();
  then_this();
  return 0;
}
```

Control Flow

Definition

We use *control flow* to describe the order instructions are executed.

In other words, how the program counter is updated.

This may be *implicit*, such as as part of the execution cycle, or *explicit*, by the execution of an instruction with modifies the special-purpose register (e.g., branch/jump instructions).

Examples from Hex 8

We've seen explicit control flow instructions already in Hex 8:

- BR $pc \leftarrow pc + oreg$: branch relative unconditional
- BRZ if $areg = 0$ then $pc \leftarrow pc + oreg$: branch relative zero
- BRN if $areg < 0$ then $pc \leftarrow pc + oreg$: branch relative negative
- BRB $pc \leftarrow breg$: branch absolute

Note the following terms: **absolute**, **relative**, **unconditional**, **conditional**.

Absolute and Relative branches

Absolute branches set the program counter to a fixed value.

Relative branches increment the program counter by some offset: $pc \leftarrow pc + co$

Note that the offset is going to be "off by one" than what you might expect. Why?

Absolute and Relative branches

Absolute branches set the program counter to a fixed value.

Relative branches increment the program counter by some offset: $pc \leftarrow pc + co$

Note that the offset is going to be "off by one" than what you might expect. Why?

The program counter is typically incremented implicitly during the execution cycle *before* the instruction execution step.

Absolute and Relative branches

Absolute branches set the program counter to a fixed value.

Relative branches increment the program counter by some offset: $pc \leftarrow pc + co$

Note that the offset is going to be "off by one" than what you might expect. Why?

The program counter is typically incremented implicitly during the execution cycle *before* the instruction execution step.

Position independent code

Relative branches allow us to write position independent code.
Such a program can be placed anywhere in memory and still execute correctly.

Other types of branches

Compare-then-branch / compare-and-branch

Note the different between `if (b) { x();}` and `if (i > 0) { x(); }`.

The former is a *conditional* branch on some boolean value.

The later includes a *comparison* expression, and branches based on the computed boolean result.

Other types of branches

Computed

For an immediate branch, $pc \leftarrow x$, the value is given in the program text as an operand.

But the branch target (i.e., x) can be computed and come from a register/memory: $pc \leftarrow GPR[x]$.

This allows the program to calculate the branch target, and change it as the program is running.

Other types of branches

Branch-and-link

It can be convenient to "save" the value of the program counter before updating it. For example, for a function call, we want to return to the caller once the callee has finished.

A branch-and-link instruction might save the PC in another special-purpose register, and then update the PC itself.

Example: if statements

```
if ( expr ) {
  stmt;
}
```

$$
\begin{aligned}
& GPR[c] \leftarrow expr \\
& \text{if } GPR[c] = 0 \text{ then } PC \leftarrow done \\
& stmt \\
done : & \ldots
\end{aligned}
$$

Example: if-else statements

```
if ( expr ) {
  stmtA;
} else {
  stmtB;
}
```

$$
\begin{aligned}
& GPR[c] \leftarrow expr \\
& \text{if } GPR[c] = 0 \text{ then } PC \leftarrow else \\
& stmtA \\
& PC \leftarrow done \\
else :& stmtB \\
done :& \ldots
\end{aligned}
$$

Example: while and do-while statements

```
while ( expr ) {
  stmt;
}
```

$$
\begin{aligned}
&loop: && GPR[c] \leftarrow expr \\
& && \text{if } GPR[c] = 0 \text{ then } PC \leftarrow done \\
& && stmt \\
& && PC \leftarrow loop \\
&done: && \ldots
\end{aligned}
$$

```
do {
  stmt;
} while ( expr )
```

$$
\begin{aligned}
&loop: && stmt \\
& && GPR[c] \leftarrow expr \\
& && \text{if } GPR[c] \neq 0 \text{ then } PC \leftarrow loop \\
&done: && \ldots
\end{aligned}
$$

Example: for loops

```
for ( exprA ; exprB; exprC; ) {
  stmt;
}
```

$$
\begin{aligned}
& ? \leftarrow exprA \\
loop : & GPR[c] \leftarrow exprB \\
& \text{if } GPR[c] = 0 \text{ then } PC \leftarrow done \\
& stmt \\
& ? \leftarrow exprC \\
& PC \leftarrow loop \\
done : & \ldots
\end{aligned}
$$

Example: function calls with branch-and-link

```
void callee() {
  ...
  return;
}

void caller() {
  ...
  callee();
  ...
  return;
}
```

$callee$:...

$PC \leftarrow LR$

...

$caller$:...

$LR \leftarrow PC, PC \leftarrow callee$

...

$PC \leftarrow LR$

...

Control and Data paths

Datapaths

Definition: Datapath

The *datapath* consists of the hardware that updates the processor state.

Consists of:

- Registers.
- Functional units (e.g. ALU).
- Memory.
- Connections between them.

The datapath contains the logic that transforms the processor state.

Control path

Definition: Control path

The *control path* consists of the hardware that updates the *state* of the processor according to program instructions.

Control signals will choose which parts of the datapath are active based on the current instruction, special-purpose registers, etc.
Allows the datapath to be a programmable circuit.

Example: flip-flops

Recall that flip-flops have an "enable" signal which specified if new state should be stored or not.

Controlling the Execution Cycle

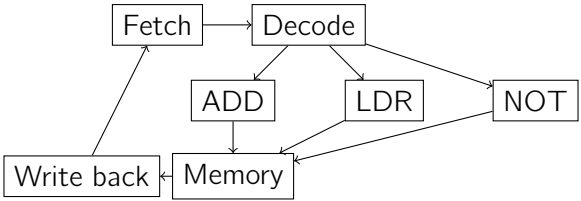
Control implements a finite state machine.

Input: Instruction.

Output: Control signals to datapath.

State: Step in execution cycle.

Recall a classic RISC Execution Cycle: Fetch, Decode, Execute, Memory, Write back.



Hardwired control

To control the datapath, the control unit has to send control signals depending on the instruction.

- Fixed functionality implemented by combinatorical logic gates.
- If a new instruction is added, need to add more *hardware* logic to change — not possible in a manufactured processor.

Can describe this as a large true table, mapping inputs (execution cycle phase, decoded instruction, etc) to output enable signals for all datapath components.

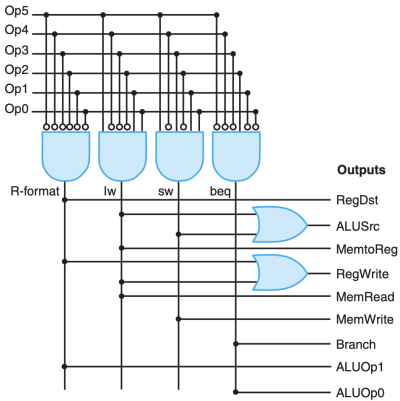


FIGURE D.2.5 The structured implementation of the control function as described by the truth table in Figure D.2.4. The structure, called a programmable logic array (PLA), uses an array of AND gates followed by an array of OR gates. The inputs to the AND gates are the function inputs and their inverses (bubbles indicate inversion of a signal). The inputs to the OR gates are the outputs of the AND gates (or, as a degenerate case, the function inputs and inverses). The output of the OR gates is the function outputs.

From Patterson and Hennessy, Computer Organization and Design

8-bit computer: Control

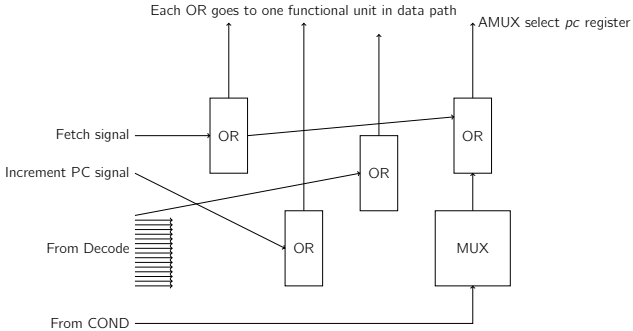
8-bit computer: Control

Use OR Module boards to implement the hardwired control unit for your 8-bit computer.

- OR Module board computes logical OR on four 1-bit signals.
- 1-bit control output set to 1 if any inputs have lowest bit set to 1.
- Input signals passed through to output: so can propagate signal on.
- Attach OR Module to each part of Data path that needs control.

8-bit computer: Control

- Connect a OR module to each functional unit of the Data Path.
- Use unique signals from Decode to control Data Path units using OR(s).
- Recall: OR unit will send 1 signal if any of inputs active.
- Use signals from Timing DEMUX for control of Fetch and PC increment.
- Use a MUX to apply ALU COND output to DEMUX signal to implement branches.
- Pass inputs through to send signal to multiple Data Path units.



This is just a sample, you'll need to use many wires and OR modules to do this in ModuleSim...

Micro-programmed control

Micro-programmed control

Alternatively, can design control unit to be *programmable* during design.

- Completely hidden from the programmer at the ISA level.
- Part of the micro-architecture.
  - Micro-instructions relate to the specific circuits in the processor.
- Micro-instructions state which control signals should be active.
- Program instructions turned into sequence of micro-instructions which execute using their own micro-program counter.

Computer inside a computer!

The need for Micro-programmed control

- CISC ISAs have complicated state machines, with instructions performing several simpler steps.
- Time consuming for designer to wire everything together correctly.
- Easier to change the design as control was programmable.

A micro-program for the Hex 8 computer

Take the SUB instruction: $areg \leftarrow areg - breg$
In our micro-architecture, need to:

- Set AMUX control to select *areg* (set control signal to $**00$, where $*$ can be either).
- Set BMUX control to select *breg* (set control signal to $**00$).
- Set ALU control to do two's complement subtraction ($areg + \text{NOT } breg + 1$) ($*110$).
- Set *rreg* control to capture ALU result ($*101$).
- Then, on next ϕ_2 clock: change *rreg* control to prevent store (such as $*100$) and set *areg* control to store ($*101$).

This micro-program describes the steps required to control the Data Path.

Micro-programmed control

Micro-program stored in read-only memory (ROM) or programmable logic array inside the processor.

- Possible to update using firmware update mechanisms.

Spectre/Meltdown security fixes on Intel processors required microcode update.

- Problem was that speculative execution in the processor (a micro-architecture optimisation) could leak information.
- Processors "guessed" branch results to save cycles.
- Micro-architecture bug fixed by updating the microcode.

More info: `https://newsroom.intel.com/news-releases/intel-issues-updates-protect-systems-security-exploits/#gs.ios4pd`

Further reading

- Wikipedia: Instruction Set Architecture (ISA). `https://en.wikipedia.org/wiki/Instruction_set_architecture`
- Wikipedia: Control flow. `https://en.wikipedia.org/wiki/Control_flow`
- D. Page. Section 5.5: Control flow. In: A Practical Introduction to Computer Architecture. 1st ed. Springer, 2009.
- A.S. Tanenbaum and T. Austin. Section 5.6: Flow of control. In: Structured Computer Organisation. 6th ed. Prentice Hall, 2012.
- W. Stallings. Chapter 12: Instruction sets: characteristics and functions. In: Computer Organisation and Architecture. 9th ed. Prentice Hall, 2013.
- Wikipedia: Position Independent Code. `https://en.wikipedia.org/wiki/Position-independent_code`

Further reading

- Patterson and Hennessy, Computer Organization and Design: The Hardware/Software Interface, Section 4.4.
- And appendix D: `https://www.elsevier.com/books-and-journals/book-companion/9780128201091`
- `https://en.wikipedia.org/wiki/Programmable_logic_array`
- Patt and Patel, Introduction to Computing Systems, Chapter 4 and Appendix C.4.
- `https://en.wikipedia.org/wiki/Classic_RISC_pipeline`
- `https://meltdownattack.com`
- `https://en.wikipedia.org/wiki/Spectre_(security_vulnerability)`

Further reading

- https://en.wikipedia.org/wiki/Control_unit
- http://fourier.eng.hmc.edu/e85_old/lectures/processor/node11.html
- https://courses.cs.washington.edu/courses/cse378/01sp/Lectures/15microprogramming.pdf
- https://en.wikipedia.org/wiki/Microcode